LUMEN PROGRAMMING LANGUAGE

VERSION 0.2.0

lumen

The Handbook

A guide and reference to a small, dynamically typed language: values, scope, functions, closures, errors and the command line.

GUIDE AND REFERENCE

EVERY EXAMPLE EXECUTED

Contents

Chapters 1 to 8 teach the language in order. Chapters 9 to 12 are for when something goes wrong, when you want worked programs, or when you want Lumen inside a Rust program. The appendices are reference.

01	Getting Started	3
02	Values and Types	6
03	Variables and Scope	9
04	Operators	12
05	Control Flow	16
06	Functions	19
07	Closures	23
08	Built-in Functions	27
09	Errors	29
10	Lines and Layout	32
11	Recipes	35
12	Embedding in Rust	40
A	Grammar	42
B	Quick Reference	44
C	Limits and Known Gaps	45

CHAPTER 01

Getting Started

Install the interpreter, run a first program four different ways, and get comfortable in the interactive prompt.

Install

Lumen is a single native executable built from Rust source. With a Rust toolchain installed, build and install it from the project directory:

```
$ cargo install --path .
$ lumen --version
Lumen 0.2.0
```

This places `lumen` in `~/.cargo/bin`. If that directory is on your `PATH`, the command works from anywhere.

A first program

Save this as `hello.lm`. The `.lm` extension is a convention, not a requirement.

```
# hello.lm
let name = "world"
print "Hello, " + name
```

OUTPUT

```
Hello, world
```

Run it by passing the file to `lumen`:

```
$ lumen hello.lm
Hello, world
```

Four ways to run code

COMMAND	WHAT IT DOES
`lumen`	Starts the interactive prompt (the REPL).
`lumen file.lm`	Runs a script.
`lumen -c 'print 6 * 7'`	Runs a program given as a string.
`lumen -`	Runs a program read from standard input. Piping into plain `lumen` does the same.
`lumen --version`	Prints the version. `-V` is the short form.
`lumen --help`	Prints usage. `-h` is the short form.

Executable scripts

Because `#` starts a comment, a Unix shebang line is valid Lumen. Mark the file executable and it runs like any other program:

```
$ head -1 examples/closures.lm
#!/usr/bin/env lumen
$ chmod +x examples/closures.lm
$ ./examples/closures.lm
```

The interactive prompt

Run `lumen` with no arguments to explore the language one entry at a time.

```
$ lumen
Lumen 0.2.0 (linux x86_64)
Ctrl-D to exit.
> let x = 21
> fn twice(n) {
...     return n * 2
... }
> twice(x)
42
> print nope
[line 1] RuntimeError: undefined variable 'nope'
> x + 1
22
```

- While a `{` or `(` is still open, the prompt changes to `...` and keeps reading. That is how multi-line blocks are entered.
- A bare expression is evaluated and its value echoed, unless the value is `nil`.

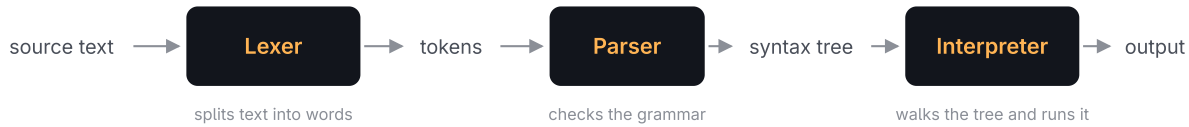
- Everything you define stays defined. An error reports and returns you to the prompt.
- Arrow keys edit the line and recall history, which is saved in `~/.lumen_history` between sessions.
- Ctrl-C abandons a half-typed entry. Ctrl-D exits.

Exit codes

CODE	MEANING
`0`	The program ran to completion.
`1`	A syntax or runtime error occurred, or the file could not be read. The message is on standard error.
`2`	The command line itself was wrong. Usage is printed.

What happens when you run a program

Lumen reads the whole program before running any of it. A syntax error anywhere in the file therefore stops the program before its first line executes.



Lexer and parser problems are reported as `SyntaxError`. Problems while the interpreter is walking the tree are reported as `RuntimeError`.

CHAPTER 02

Values and Types

Lumen has five kinds of value. Variables do not have types; the values in them do.

TYPE	WRITTEN AS	NOTES
number	`42` `3.14`	One numeric type: a 64-bit float.
string	`"hello"`	Text in double quotes.
boolean	`true` `false`	The result of comparisons.
nil	`nil`	The absence of a value.
function	`fn(x) { return x }`	A value like any other. See chapters 6 and 7.

Numbers

There is no separate integer type. Whole numbers are exact up to 9,007,199,254,740,992 (2^{53}), and a number with no fractional part prints without a decimal point.

```
print 42
print 3.14
print 7 / 2
print 6 / 2          # a whole result prints as a whole number
print 0.1 + 0.2      # floats are floats
print 10 % 3
print -7 % 3         # the result takes the sign of the right-hand side
```

OUTPUT

```
42
3.14
3.5
3
0.30000000000000004
1
2
```

A number literal is digits, optionally followed by a point and more digits. There is no exponent form, and `.5` and `5.` are not numbers; write `0.5` and `5`.

Strings

Strings are written in double quotes and may contain four escape sequences.

ESCAPE	PRODUCES
`\n`	a new line
`\t`	a tab
`\"`	a double quote
`\\`	a backslash

```
print "Hello" + ", " + "world"
print "She said \"hi\""
print "one\ntwo"
print "total: " + 42
print len("lumen")
```

OUTPUT

```
Hello, world
She said "hi"
one
two
total: 42
5
```

The `+` operator joins strings. If only one side is a string, the other side is converted to text first, which is how `"total: " + 42` works. That is also the way to turn any value into a string: add it to `""`.

> **NOTE**
> Strings cannot be indexed or sliced, and there is no way to take one apart character by character. `len` and `+` are the string operations.

Booleans and nil

`true` and `false` come from comparisons and the logical operators. `nil` is what you get from a variable declared without a value, or from a function that does not return anything.

Truthiness

Any value can be used as a condition. The rule is short: **only `false` and `nil` are falsy**. Everything else counts as true, including zero and the empty string.

```
if 0 { print "0 is truthy" }
if "" { print "the empty string is truthy" }
if nil { print "this line never prints" }
if not nil { print "nil is falsy" }
```

OUTPUT

```
0 is truthy
the empty string is truthy
nil is falsy
```

Equality

`==` and `!=` work on any two values and never raise an error. Values of different types are never equal, and nothing is converted to make them match.

```
print 1 == 1
print "a" == "a"
print "1" == 1       # a string is never equal to a number
print nil == false   # nil is only equal to nil
print 0 == false
```

OUTPUT

```
true
true
false
false
false
```

Two function values are equal only when they are the very same function.

CHAPTER 03

Variables and Scope

Names are introduced with `let`, changed with `=`, and visible only inside the block that declares them.

Declaring and assigning

```
let name = "Ada"
let age              # no initial value: starts as nil
print age
age = 36
print name + " is " + age
```

```
OUTPUT
nil
Ada is 36
```

A name is letters, digits and underscores, and cannot start with a digit. The thirteen keywords listed in Appendix B cannot be used as names.

`let` creates a variable. `=` changes one that already exists. Assigning to a name that was never declared is an error, which catches typos that would otherwise silently create a new variable.

```
let count = 1
cuont = 2
```

```
ERROR
[line 2] RuntimeError: cannot assign to undefined variable 'cuont'
```

Assignment is an expression: it produces the assigned value, and chains from right to left.

```
let a = 0
let b = 0
a = b = 7
print a + b
```

```
OUTPUT
14
```

Blocks

A pair of braces is a block, and every block is a new scope. Variables declared inside disappear when the block ends. A block can stand on its own, and it is also what follows `if`, `while` and `fn`.

```
{
    let secret = 42
    print secret
}
print secret
```

OUTPUT

```
42
```

ERROR

```
[line 5] RuntimeError: undefined variable 'secret'
```

Shadowing

An inner block may declare a variable with the same name as an outer one. The inner name hides the outer one until the block ends; the outer variable is untouched.

```
let x = "outer"
{
    let x = "inner"
    print x
}
print x
```

OUTPUT

```
inner
outer
```

Reaching outward

Without `let`, an assignment finds the nearest enclosing variable of that name and changes it.

```
let total = 0
{
    total = total + 10    # no `let`: this is the outer total
}
print total
```

OUTPUT

```
10
```

CAREFUL

The difference between `let total = ...` and `total = ...` inside a block is the difference between a new variable and changing the old one. Inside a loop body, a stray `let` gives you a fresh variable on every pass.

Scope follows the text

Lumen is lexically scoped. A function sees the variables that were visible where it was *written*, not the ones visible where it is *called*.

```
let x = "global"

fn show() {
    return x
}

fn test() {
    let x = "local"     # show() does not see this one
    return show()
}

print test()
```

OUTPUT

```
global
```

This one rule is what makes closures, the subject of chapter 7, work.

CHAPTER 04

Operators

Arithmetic, comparison and logic, and the order in which they are applied.

Precedence

Operators lower in this table bind more tightly. All binary operators group from the left, so `10 - 4 - 3` is `(10 - 4) - 3`. Parentheses override everything.

LEVEL	OPERATORS	MEANING
1 (loosest)	`=`	Assignment. Groups from the right.
2	`or`	Logical or.
3	`and`	Logical and.
4	`not`	Logical negation, written before its operand.
5	`==` `!=`	Equal, not equal.
6	`<` `<=` `>` `>=`	Ordering.
7	`+` `-`	Add or join, subtract.
8	`*` `/` `%`	Multiply, divide, remainder.
9	`-`	Negation, written before its operand.
10 (tightest)	`f(x)`	Function call.

```
print 1 + 2 * 3        # multiplication first
print (1 + 2) * 3
print 10 - 4 - 3       # grouped from the left
print -2 * 3
print 1 + 2 < 4        # arithmetic before comparison
print not 1 == 2       # comparison before `not`
print true or false and false   # `and` before `or`
```

```
OUTPUT
7
9
3
-6
true
true
true
```

Arithmetic

`-`, `*`, `/` and `%` require two numbers. `/` is always floating-point division. `%` is the remainder, with the sign of its right-hand side. Dividing or taking a remainder by zero is an error.

```
let parts = 0
print 100 / parts
```

```
ERROR
[line 2] RuntimeError: division by zero
```

To divide and discard the remainder, subtract the remainder first:

```
let n = 17
print (n - n % 5) / 5
```

```
OUTPUT
3
```

Comparison

`<`, `<=`, `>` and `>=` compare two numbers, or two strings in dictionary order. Mixing the two is an error.

```
print 3 < 10
print "apple" < "banana"
print "Zebra" < "apple"    # capitals sort before lower case
```

OUTPUT

```
true
true
true
```

Logic

`and` and `or` stop as soon as the answer is known, and they give back one of their operands rather than a plain `true` or `false`.

- `a or b` is `a` if `a` is truthy, otherwise `b`.
- `a and b` is `a` if `a` is falsy, otherwise `b`.

```
print nil or "default"          # a fallback value
print "first" and "second"
print false and missing()       # missing() is never called
print not "text"
```

OUTPUT

```
default
second
false
false
```

The third line does not fail even though no function named `missing` exists: the right-hand side is never evaluated. `not` always gives a boolean.

When types do not fit

Lumen does not guess. Apart from `+` with a string and `==`, an operator given the wrong kind of value stops the program and names both types.

```
print "10" - 1
```

```
ERROR
[line 1] RuntimeError: unsupported operands for '-': string and number
```

CHAPTER 05

Control Flow

Two statements steer a program: `if` *chooses, and* `while` *repeats.*

if, else if, else

The condition is written without parentheses, and the braces are always required, even around a single statement.

```
let temperature = 31

if temperature > 30 {
    print "hot"
} else if temperature > 15 {
    print "mild"
} else {
    print "cold"
}
```

OUTPUT

```
hot
```

Any value can be the condition; chapter 2 gives the truthiness rule. A short body can sit on one line:

```
let name = ""
if len(name) == 0 { name = "anonymous" }
print name
```

OUTPUT

```
anonymous
```

while

`while` evaluates its condition, runs the block if the condition is truthy, and repeats. It is the only loop.

```
let n = 3
while n > 0 {
    print n
    n = n - 1
}
print "liftoff"
```

OUTPUT

```
3
2
1
liftoff
```

Counting and accumulating

The usual pattern is a counter declared before the loop and advanced at the end of the body. Here the results are gathered into one string and printed once.

```
let i = 1
let line = ""
while i <= 15 {
    let word = i
    if i % 15 == 0 {
        word = "FizzBuzz"
    } else if i % 3 == 0 {
        word = "Fizz"
    } else if i % 5 == 0 {
        word = "Buzz"
    }
    line = line + word + " "
    i = i + 1
}
print line
```

OUTPUT

```
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

Note where each variable lives. `i` and `line` are declared outside the loop, so they persist. `word` is declared inside the body, so each pass gets a fresh one.

Leaving a loop early

There is no `break` or `continue`. To stop in the middle of a loop, put the loop in a function and `return`.

```
fn first_multiple(of, above) {
    let n = above + 1
    while true {
        if n % of == 0 { return n }
        n = n + 1
    }
}

print first_multiple(7, 100)
```

OUTPUT

```
105
```

To skip the rest of one pass, wrap the rest of the body in an `if`.

CAREFUL

A `while` whose condition never becomes falsy runs forever. From the command line, Ctrl-C stops it.

CHAPTER 06

Functions

Functions package up behaviour. In Lumen they are also ordinary values: they can be stored, passed to other functions, and returned.

Declaring and calling

```
fn area(width, height) {
    return width * height
}

print area(3, 4)
```

OUTPUT

```
12
```

`return` ends the function and hands back a value. A function that reaches the end of its body, or uses `return` with nothing after it, gives back `nil`.

```
fn greet(name) {
    print "Hello, " + name
}

let result = greet("Ada")
print result
```

OUTPUT

```
Hello, Ada
nil
```

A call must supply exactly as many arguments as the function has parameters.

```
fn area(width, height) {
    return width * height
}
print area(3)
```

```
ERROR
[line 4] RuntimeError: <fn area> expects 2 argument(s), got 1
```

Functions are values

The word `fn` without a name creates a function on the spot. In fact `fn area(...) {...}` is only a shorter way of writing `let area = fn(...) {...}`.

```
let double = fn(x) { return x * 2 }
print double(21)
print double
print fn(x) { return x * x }(9)     # created and called at once
```

```
OUTPUT
42
<fn>
81
```

Passing functions

```
fn apply_twice(f, value) {
    return f(f(value))
}

fn add_ten(n) { return n + 10 }

print apply_twice(add_ten, 1)
print apply_twice(fn(s) { return s + "!" }, "wow")
```

```
OUTPUT
21
wow!!
```

Returning functions

```
fn compose(f, g) {
    return fn(x) { return f(g(x)) }
}

let inc = fn(x) { return x + 1 }
let square = fn(x) { return x * x }

print compose(square, inc)(4)    # square(inc(4))
print compose(inc, square)(4)    # inc(square(4))
```

```
OUTPUT
25
17
```

Recursion

A function may call itself. Every recursive function needs a case that does not recurse, or it will never finish.

```
fn factorial(n) {
    if n <= 1 { return 1 }
    return n * factorial(n - 1)
}

fn fib(n) {
    if n < 2 { return n }
    return fib(n - 1) + fib(n - 2)
}

print factorial(10)
print fib(20)
```

```
OUTPUT
3628800
6765
```

Functions can also call each other, in either order, because a name is looked up when the call happens, not when the function is written.

```
fn is_even(n) {
    if n == 0 { return true }
    return is_odd(n - 1)
}

fn is_odd(n) {
    if n == 0 { return false }
    return is_even(n - 1)
}

print is_even(10)
print is_even(7)
```

OUTPUT

```
true
false
```

The depth limit

Calls may nest 10,000 deep. Beyond that the program stops with a clear error instead of crashing.

```
fn forever(n) {
    return forever(n + 1)
}
forever(0)
```

ERROR

```
[line 2] RuntimeError: stack overflow
```

NOTE

There is no tail-call optimisation, so a loop that runs more than 10,000 times should be a `while`, not a recursion.

CHAPTER 07

Closures

A function remembers the variables that were in scope where it was created, and keeps them alive. That single idea gives Lumen private state, configurable functions, and even data structures.

A function that remembers

```
fn make_counter() {
    let count = 0
    return fn() {
        count = count + 1
        return count
    }
}

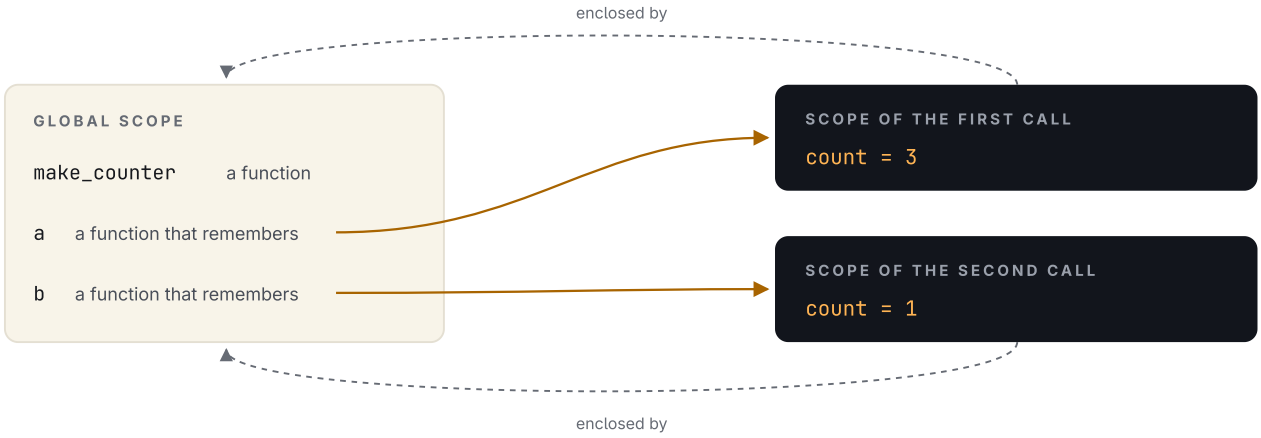
let a = make_counter()
let b = make_counter()

a()
a()
print "a: " + a()
print "b: " + b()
```

OUTPUT

```
a: 3
b: 1
```

`make_counter` has returned, yet `count` lives on, because the function it returned still refers to it. Each call to `make_counter` creates a new `count`, so `a` and `b` count separately. Nothing outside can reach either `count` except through its function.



After the program above has run. Each counter function holds on to the scope of the call that created it, and each scope has its own `count`.

The variable, not a copy

A closure holds the variable itself. If the variable changes later, the closure sees the new value.

```
let greeting = "Hello"
let greet = fn(name) { return greeting + ", " + name }

print greet("Ada")
greeting = "Goodbye"
print greet("Ada")
```

OUTPUT

```
Hello, Ada
Goodbye, Ada
```

Configured functions

A function that takes a setting and returns a function is a cheap way to make a family of related functions.

```
fn multiplier(factor) {
    return fn(x) { return x * factor }
}

let double = multiplier(2)
let percent = multiplier(0.01)

print double(21)
print percent(250)
```

```
OUTPUT
42
2.5
```

Private state

A closure can guard its data and decide what is allowed. Here the balance can only be changed through the function, and the function refuses to overdraw.

```
fn make_account(balance) {
    return fn(action, amount) {
        if action == "deposit" {
            balance = balance + amount
        }
        if action == "withdraw" {
            if amount > balance { return "insufficient funds" }
            balance = balance - amount
        }
        return balance
    }
}

let account = make_account(100)
print account("deposit", 50)
print account("withdraw", 30)
print account("withdraw", 500)
print account("balance", 0)
```

```
OUTPUT
150
120
insufficient funds
120
```

Data from closures

Lumen has no lists. A closure that remembers two values is enough to build one: each cell holds an item and the rest of the list, and `nil` marks the end.

```
fn cons(head, tail) {
    return fn(want_head) {
        if want_head { return head }
        return tail
    }
}
fn head(list) { return list(true) }
fn tail(list) { return list(false) }

fn range(from, to) {
    if from > to { return nil }
    return cons(from, range(from + 1, to))
}

fn map(f, list) {
    if list == nil { return nil }
    return cons(f(head(list)), map(f, tail(list)))
}

fn sum(list) {
    if list == nil { return 0 }
    return head(list) + sum(tail(list))
}

fn show(list) {
    let text = "["
    while list != nil {
        text = text + head(list)
        list = tail(list)
        if list != nil { text = text + ", " }
    }
    return text + "]"
}

print show(range(1, 6))
print show(map(fn(x) { return x * x }, range(1, 6)))
print sum(range(1, 100))
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
[1, 4, 9, 16, 25, 36]
5050
```

Everything in this program is functions and closures: the list cells, the operations on them, and the squaring function handed to `map`.

CHAPTER 08

Built-in Functions

Two functions are available everywhere without being declared.

FUNCTION	ARGUMENTS	RETURNS
`len(s)`	one string	The number of characters in `s`.
`clock()`	none	The current time as seconds since 1 January 1970 (UTC), with a fractional part.

len

```
print len("lumen")
print len("")
print len("naïve café")       # characters, not bytes
```

OUTPUT

```
5
0
10
```

`len` accepts only strings.

```
print len(12345)
```

ERROR

```
[line 1] RuntimeError: len() expects a string, got number
```

To count the digits of a number, turn it into a string first:

```
print len("" + 12345)
```

OUTPUT

```
5
```

clock

The absolute value of `clock()` is rarely interesting. The difference between two readings is: it tells you how long something took.

```
fn fib(n) {
    if n < 2 { return n }
    return fib(n - 1) + fib(n - 2)
}

let start = clock()
let result = fib(22)
let elapsed = clock() - start

print "fib(22) = " + result
print "finished in under a second: " + (elapsed < 1)
```

OUTPUT

```
fib(22) = 17711
finished in under a second: true
```

Built-ins are ordinary values

They can be stored, passed around and printed like any function. They can also be shadowed: a `let len = ...` of your own hides the built-in within that scope.

```
let measure = len
print measure("four")
print len
print clock == clock
```

OUTPUT

```
4
<builtin fn len>
true
```

CHAPTER 09

Errors

When Lumen cannot continue, it says what went wrong and on which line. There are two kinds of error, and they happen at different times.

Reading an error

```
[line 3] RuntimeError: undefined variable 'totl'
 ------  ------------  ------------------------
 where   which kind    what happened
```

Errors are written to standard error, and the process exits with code 1.

Syntax errors

A syntax error means the text is not a valid program. The whole file is checked before anything runs, so a syntax error on the last line prevents the first line from running.

```
print "this never prints"
let total = (1 + 2
```

```
ERROR
[line 2] SyntaxError: expected ')', found end of input
```

MESSAGE	CAUSE
`expected expression, found ...`	A value was needed and something else was there, such as after `let x =` or a trailing operator.
`expected '{'` / `'}'` / `')'`	A bracket is missing. Missing closers are reported where the parser gave up, often the end of the file.
`expected variable name`	`let` was not followed by a name, or the name is a keyword.
`expected parameter name`	Something other than a name appeared in a parameter list.
`expected newline or ';' after statement`	Two statements share a line without a `;` between them.
`invalid assignment target`	The left of `=` is not a variable name, as in `a + b = 3`.

`'return' outside of a function`	`return` at the top level of a program.
`duplicate parameter 'x'`	Two parameters of one function share a name.
`unterminated string`	A string's closing quote is missing. The line is where the string began.
`invalid escape '\q'`	A backslash followed by anything other than `n`, `t`, `"` or `\`.
`unexpected character '@'`	A character that is not part of the language, outside a string or comment.

Runtime errors

A runtime error happens while the program is running. Everything before the failing line has already taken effect, and its output has already been printed.

```
print "step 1"
print "step 2"
print missing_value
print "step 3"
```

```
OUTPUT
step 1
step 2
```

```
ERROR
[line 3] RuntimeError: undefined variable 'missing_value'
```

MESSAGE	CAUSE
`undefined variable 'x'`	Reading a name that is not declared in any enclosing scope.
`cannot assign to undefined variable 'x'`	Assigning without a prior `let`.
`unsupported operands for '-': string and number`	An operator was given types it does not accept. Both types are named.
`operand of unary '-' must be a number, got string`	Negating something that is not a number.
`division by zero` / `modulo by zero`	The right-hand side of `/` or `%` was zero.
`can only call functions, not string`	Call parentheses after a value that is not a function.
`<fn f> expects 2 argument(s), got 1`	The number of arguments does not match the number of parameters.

`len() expects a string, got number`	A built-in was given the wrong type.
`stack overflow`	More than 10,000 nested calls, almost always a recursion with no stopping case.

Which line is reported

The line is that of the operator, the opening parenthesis of the call, or the variable name involved. When the error is inside a function, it is the line inside the function, not the line that called it.

```
fn average(total, count) {
    return total / count
}

print average(10, 4)
print average(10, 0)
```

```
OUTPUT
2.5
```

```
ERROR
[line 2] RuntimeError: division by zero
```

Working without exceptions

Lumen has no way to catch an error: a runtime error ends the program. Check before you act, and return `nil` or a message when a function cannot do its job.

```
fn safe_divide(a, b) {
    if b == 0 { return nil }
    return a / b
}

let answer = safe_divide(10, 0)
print answer or "cannot divide by zero"
```

```
OUTPUT
cannot divide by zero
```

CHAPTER 10

Lines and Layout

Lumen has no statement terminator. A handful of rules about line breaks make that work, and each is worth knowing once.

One statement per line

A statement ends at the end of its line. To put two on one line, separate them with a semicolon. The semicolon is never needed otherwise.

```
let a = 1; let b = 2
print a + b
```

OUTPUT

```
3
```

```
let a = 1 let b = 2
```

ERROR

```
[line 1] SyntaxError: expected newline or ';' after statement, found 'let'
```

Statements that end in a block need no separator, which is why `if x { ... } else { ... }` and one-line functions read naturally.

Continuing an expression

An expression carries on to the next line whenever it is plainly unfinished: after an operator, inside parentheses, or when the next line begins with a binary operator.

```
let trailing = 1 +
    2 +
    3

let leading = 1
    + 2
    + 3

print trailing
print leading
```

```
OUTPUT
6
6
```

Three rules about the same line

A call's parenthesis stays with the function

The `(` that begins an argument list must be on the same line as what it calls. A `(` at the start of a line always begins a new statement.

```
fn five() { return 5 }

let f = five
(1 + 2)          # a separate statement, not a call to five

print f
```

```
OUTPUT
<fn five>
```

A return value starts on the return line

If nothing follows `return` on its own line, the function returns `nil`, and whatever is on the next line is a separate statement.

```
fn oops() {
    return
    42
}

print oops()
```

OUTPUT

```
nil
```

else follows the closing brace

In a file, `else` may be on the line after the `}`. In the interactive prompt it must be on the same line, because a complete entry is run as soon as its brackets balance.

> CAREFUL
> A line that begins with `-` continues the line before it, because `-` is also a binary operator. `let a = 1` followed by a line reading `-a` is parsed as `let a = 1 - a`. If a statement must begin with a minus sign, end the previous line with a semicolon.

Comments and blank space

A `#` begins a comment that runs to the end of the line. Indentation and blank lines are ignored; four spaces per level is the convention used in this handbook. A string may contain a line break, which becomes part of the string.

```
# A whole-line comment.
let poem = "roses are red
violets are blue"      # a comment after code
print poem
```

OUTPUT

```
roses are red
violets are blue
```

CHAPTER 11

Recipes

Short, complete programs for common jobs. Each one runs as printed.

Smallest, largest, absolute value

```
fn min(a, b) { if a < b { return a } return b }
fn max(a, b) { if a > b { return a } return b }
fn abs(n) { if n < 0 { return -n } return n }

print min(3, 8)
print max(3, 8)
print abs(-12.5)
```

OUTPUT

```
3
8
12.5
```

Whole-number division and rounding

```
fn floor(n) { return n - n % 1 }
fn div(a, b) { return floor(a / b) }
fn round(n) { return floor(n + 0.5) }

print floor(7.9)
print div(17, 5)
print round(2.5)
print round(2.49)
```

OUTPUT

```
7
3
3
2
```

Greatest common divisor

```
fn gcd(a, b) {
    while b != 0 {
        let rest = a % b
        a = b
        b = rest
    }
    return a
}

fn lcm(a, b) { return a / gcd(a, b) * b }

print gcd(1071, 462)
print lcm(4, 6)
```

OUTPUT

```
21
12
```

Powers

Squaring halves the work at every step, so even large exponents take few multiplications.

```
fn pow(base, exp) {
    if exp == 0 { return 1 }
    let half = pow(base, (exp - exp % 2) / 2)
    if exp % 2 == 0 { return half * half }
    return half * half * base
}

print pow(2, 10)
print pow(3, 13)
print pow(1.5, 2)
```

OUTPUT

```
1024
1594323
2.25
```

Prime numbers

```
fn is_prime(n) {
    if n < 2 { return false }
    let d = 2
    while d * d <= n {
        if n % d == 0 { return false }
        d = d + 1
    }
    return true
}

let n = 1
let found = ""
while n < 50 {
    if is_prime(n) { found = found + n + " " }
    n = n + 1
}
print found
```

OUTPUT

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

Repeating and padding text

```
fn repeat(text, times) {
    let out = ""
    while times > 0 {
        out = out + text
        times = times - 1
    }
    return out
}

fn pad_left(value, width) {
    let text = "" + value
    return repeat(" ", width - len(text)) + text
}

print repeat("=-", 12)
print pad_left(7, 6) + pad_left(42, 6) + pad_left(1234, 6)
print pad_left(100, 6) + pad_left(3.5, 6) + pad_left(0, 6)
```

OUTPUT

```
=-=-=-=-=-=-=-=-=-=-=-=-
     7    42  1234
   100   3.5     0
```

Number to binary

```
fn to_binary(n) {
    if n < 2 { return "" + n }
    return to_binary((n - n % 2) / 2) + n % 2
}

print to_binary(5)
print to_binary(255)
print to_binary(1024)
```

```
OUTPUT
101
11111111
10000000000
```

Run something n times

A function that takes a function is Lumen's substitute for a `for` loop.

```
fn times(n, action) {
    let i = 1
    while i <= n {
        action(i)
        i = i + 1
    }
}

let total = 0
times(100, fn(i) { total = total + i })
print total

times(3, fn(i) { print "pass " + i })
```

```
OUTPUT
5050
pass 1
pass 2
pass 3
```

Towers of Hanoi

Recursion decides the moves; a callback decides what to do with each one. The same solver first prints the moves, then merely counts them.

```
fn hanoi(n, from, to, via, on_move) {
    if n == 0 { return }
    hanoi(n - 1, from, via, to, on_move)
    on_move(from, to)
    hanoi(n - 1, via, to, from, on_move)
}

hanoi(2, "A", "C", "B", fn(from, to) { print from + " -> " + to })

let moves = 0
hanoi(10, "A", "C", "B", fn(from, to) { moves = moves + 1 })
print "10 discs: " + moves + " moves"
```

OUTPUT

```
A -> B
A -> C
B -> C
10 discs: 1023 moves
```

CHAPTER 12

Embedding in Rust

The interpreter is also a Rust library, so a Rust program can run Lumen code and read the results.

Running code

Add the crate as a dependency, create an `Interpreter`, and call `run`. Global variables and functions persist from one `run` to the next, exactly as they do in the interactive prompt.

```
use lumen::{Interpreter, LumenError};

let mut interp = Interpreter::new(); // `print` goes to stdout
interp.run("let greeting = \"2 + 2 = \"").unwrap();
interp.run("print greeting + (2 + 2)").unwrap(); // globals persist between runs

match interp.run("print 1 / 0") {
    Err(LumenError::Runtime(e)) => {
        assert_eq!((e.line, e.message.as_str()), (1, "division by zero"))
    }
    other => panic!("expected a runtime error, got {other:?}"),
}
```

The API

ITEM	PURPOSE
`Interpreter::new()`	An interpreter whose `print` writes to standard output.
`Interpreter::with_output(f)`	An interpreter that hands each printed line to the closure `f` instead.
`interp.run(source)`	Runs source text. Returns `Ok(Some(value))` if the last statement was an expression, `Ok(None)` otherwise, or `Err(LumenError)`.
`Value`	A Lumen value: `Nil`, `Bool`, `Num`, `Str`, `Fn` or `Builtin`. It implements `Display` the way `print` shows it.
`LumenError`	Either `Syntax(LumenSyntaxError)` or `Runtime(LumenRuntimeError)`. Both carry `line` and `message`.

`Session`	The prompt's line-by-line logic: `feed(line)` returns `None` while brackets are open, then the result of the whole entry.
`tokenize`, `parse`	The lexer and parser on their own, for tools that inspect programs without running them.
`run_on_big_stack(f)`	Runs `f` on a thread with a 256 MiB stack.

Capturing output

```
use std::{cell::RefCell, rc::Rc};
use lumen::Interpreter;

let lines = Rc::new(RefCell::new(Vec::new()));
let sink = lines.clone();
let mut interp = Interpreter::with_output(move |line|
sink.borrow_mut().push(line.to_string()));

interp.run("print 6 * 7").unwrap();
assert_eq!(*lines.borrow(), ["42"]);
```

> **CAREFUL**
>
> The interpreter recurses on the native stack. A deeply recursive Lumen program can exhaust a default-sized thread stack before it reaches the 10,000-call limit, which aborts the host process. Wrap the work in `lumen::run_on_big_stack(|| ...)`, as the `lumen` command does.

APPENDIX A

Grammar

The complete syntax, in EBNF. Braces mean "zero or more", square brackets mean "optional", and a bar separates alternatives.

Statements

```
program     = { statement } ;

statement   = letStmt | fnDecl | printStmt | ifStmt
            | whileStmt | returnStmt | block | exprStmt ;

letStmt     = "let" IDENT [ "=" expression ] ;
fnDecl      = "fn" IDENT "(" [ params ] ")" block ;
printStmt   = "print" expression ;
ifStmt      = "if" expression block [ "else" ( ifStmt | block ) ] ;
whileStmt   = "while" expression block ;
returnStmt  = "return" [ expression ] ;
block       = "{" { statement } "}" ;
exprStmt    = expression ;

params      = IDENT { "," IDENT } ;
```

Expressions

```
expression  = assignment ;
assignment  = IDENT "=" assignment | logicOr ;
logicOr     = logicAnd   { "or"  logicAnd } ;
logicAnd    = equality   { "and" equality } ;
equality    = comparison { ( "==" | "!=" ) comparison } ;
comparison  = term       { ( "<" | "<=" | ">" | ">=" ) term } ;
term        = factor     { ( "+" | "-" ) factor } ;
factor      = unary      { ( "*" | "/" | "%" ) unary } ;
unary       = "-" unary | "not" equality | call ;
call        = primary { "(" [ arguments ] ")" } ;
arguments   = expression { "," expression } ;

primary     = NUMBER | STRING | "true" | "false" | "nil" | IDENT
            | "(" expression ")"
            | "fn" "(" [ params ] ")" block ;
```

Tokens

```
NUMBER      = DIGIT { DIGIT } [ "." DIGIT { DIGIT } ] ;
STRING      = '"' { any character except '"' and '\' | ESCAPE } '"' ;
ESCAPE      = "\n" | "\t" | '\"' | "\\" ;
IDENT       = ( LETTER | "_" ) { LETTER | DIGIT | "_" } ;   (not a keyword)
COMMENT     = "#" { any character except a line break } ;
```

Letters are `a` to `z` and `A` to `Z`. Statements are separated by line breaks or `;`, under the rules of chapter 10, which the grammar above does not express.

APPENDIX B

Quick Reference

The whole language on one page.

KEYWORDS

`let` `fn` `return` `if` `else` `while` `print` `true` `false` `nil` `and` `or` `not`

STATEMENTS

```
let x = 1
x = x + 1
print x
if x > 1 { } else if x == 1 { } else { }
while x > 0 { x = x - 1 }
fn name(a, b) { return a + b }
return
{ let scoped = 1 }
```

EXPRESSIONS

```
42   3.14   "text"   true   false   nil
a + b   a - b   a * b   a / b   a % b
a == b   a != b
a < b   a <= b   a > b   a >= b
a and b   a or b   not a   -a
f(x, y)
fn(x) { return x }
( a + b ) * c
```

RULES TO REMEMBER

- Only `false` and `nil` are falsy.
- `+` joins when either side is a string.
- Different types are never `==`.
- `let` declares; `=` needs an existing name.
- Braces are required; conditions take no parentheses.
- One statement per line, or separate with `;`.

BUILT-INS AND ESCAPES

`len(s)` characters in a string

`clock()` seconds since 1970

`\n` new line `\t` tab

`\"` quote `\\` backslash

`#` comment to end of line

COMMAND LINE

```
lumen                start the interactive prompt
lumen file.lm        run a script
lumen -c CODE        run a program passed as a string
lumen -              run a program read from standard input (also when input is piped)
lumen -V, --version  print the version
lumen -h, --help     print usage

exit codes           0 success    1 Lumen error or unreadable file    2 bad usage
```

APPENDIX C

Limits and Known Gaps

What Lumen 0.2.0 does not do, stated plainly, with the usual way around each.

LIMIT	DETAIL AND WORKAROUND
No lists or maps	Functions are the only compound value. Build structures from closures, as in chapter 7.
No string indexing	Strings can be joined, compared and measured, but not sliced or searched.
No `break`, `continue` or `for`	Put the loop in a function and `return`; see chapters 5 and 11.
No error handling	A runtime error ends the program. Check first and return `nil`; see chapter 9.
No input	A program cannot read the keyboard, files or its command-line arguments.
No modules	A program is one file.
Call depth of 10,000	No tail-call optimisation. Use `while` for long repetition.
Number literals	No exponent form; a digit is required on both sides of the point.
Whole numbers above 2^{53}	All numbers are 64-bit floats, so very large whole numbers lose exactness.
One error at a time	Only the first syntax error in a file is reported.
Leading minus	A line beginning with `-` continues the previous line; see chapter 10.
Ctrl-C while running	Interrupting a running program ends `lumen`, including the interactive prompt.
Memory	There is no garbage collector. A closure stored in the scope it captured is not freed until the program exits.
Prompt line numbers	In the interactive prompt, line numbers in errors count from the start of the current entry.

The Lumen Handbook, for Lumen 0.2.0. Every program printed in a dark panel was run by the `lumen` executable when this document was built, and the output and error panels show exactly what it produced. Terminal transcripts and Rust listings were checked by hand. Set in P052, Adwaita Sans and JetBrains Mono.